ใบงานครั้งที่ 1 ส่วน B — วางแผนก่อนทดสอบ

ENGSE601 • Mini Evidence Pack • TeamName: TheOneBakery • Reconstructed from supplied S1 worksheet

จัดรูปแบบใหม่จากใบงานครั้งที่ 1 ส่วน B ที่ผู้ใช้ให้มา โดยตัดหน้าส่วน A และลบรหัสนักศึกษาจากตารางสมาชิก. Component flow marked "สร้างย้อนหลัง" จาก code paths ที่ตรวจบน public repo; ข้อค้นพบเป็น static code review ไม่ใช่ผล runtime.

ข้อมูล feature

หัวข้อ	ข้อมูล
TeamName	TheOneBakery (ใช้แทน "Team the one bakery" เพื่อให้ชื่อไฟล์ไม่มีช่องว่าง)
Project	THE ONE Bakery
Feature	การบันทึกคำสั่งซื้อ
Critical rule from worksheet	เมื่อมีการสั่งซื้อ ระบบต้องบันทึกข้อมูลคำสั่งซื้อและคำนวณยอดรวมให้ถูกต้อง. Formal SRS file/version referenced in the worksheet was not supplied.
Why selected	ออเดอร์และยอดรวมเป็นข้อมูลหลักที่ส่งต่อไปงาน admin/production/packing; ความผิดพลาดมีผลต่อการรับออเดอร์และการผลิต.

ขั้นที่ 2 — Component / Data Flow (สร้างย้อนหลัง)

Order	Component / layer	Data / responsibility
1	Customer UI: public/index.html + public/cart.html	ลูกค้าเลือกสินค้า/จำนวน; cart แสดงยอดเบื้องต้น
2	HTTP API: POST /api/orders	ส่ง customer_name, phone, items ไป server
3	Backend: routes/orders.js	ตรวจสินค้า active, คำนวณ total จาก products.price, เช็กกำลังผลิต/วัตถุดิบ, INSERT order/items/history/notification ภายใน transaction
4	Data layer: config/db.js + PostgreSQL schema	products, orders, order_items, recipes, inventory, order_status_history, notifications
5	Admin / kitchen work flow	ออเดอร์เริ่ม pending_review; admin review แล้วจึงเข้าขั้นตอนถัดไป

ที่มาภาพ: สร้างย้อนหลังเมื่อ 23 ต.ค. 2569 จากการตรวจ code paths ใน the-one-bakery-New/main; ไม่ใช่แผนผังที่มีอยู่ก่อนพัฒนา และไม่ได้ยืนยันการทำงานจริงของทุกจุด.

ขั้นที่ 3 — คำถามสามข้อ

คำถาม	คำตอบและเหตุผล
1. ใครรับผิดชอบ?	routes/orders.js ฝั่ง Backend คำนวณยอดรวมจาก products.price ที่อ่านจากฐานข้อมูลและบันทึก orders/order_items. UI คำนวณยอดเพื่อแสดงผลได้ แต่ไม่ควรเป็นแหล่งราคาที่เชื่อถือได้.
2. ข้อมูลส่งถึงครบไหม?	โค้ดส่ง customer_name, phone และ items ไปยัง API; route อ่าน product IDs, quantity แล้วบันทึก order และ order_items. ต้องทดสอบ runtime เพื่อยืนยันว่าทุก field และรายการ persist ครบ; ขณะนี้ยังไม่มี DB/API evidence.
3. ถ้าพัง จะรู้ได้ไหม?	บาง error ถูกตอบเป็น JSON พร้อม error/details; transaction rollback ใน catch. แต่ยังต้องทดสอบ error paths จริงและตรวจว่าข้อความ/สถานะ HTTP เหมาะสม.

ขั้นที่ 4 — ข้อค้นพบจาก static code review

ส่วน	บันทึก
1. เอกสาร/โค้ดและตำแหน่ง	routes/orders.js: cancellation route r.post('/:id/cancel', async ...) does not show auth middleware, and handler does not show ownership check; other status/decision routes have auth/allow.
2. ขัดกับข้อใด	ยังไม่พบ formal SRS/security requirement ที่อนุมัติ จึงไม่อ้างว่า violation ของ BR ID ใด; เป็น risk เรื่อง authorization ที่ต้องให้เจ้าของ requirement ยืนยัน.
3. ผลต่อการทดสอบ	ควรมีกรณีทดสอบ cancellation request โดยไม่มี credential, ของ order คนอื่น, สถานะ pending_review/confirmed และหลังเริ่มผลิต. หากใช้ public order ID อย่างเดียว อาจมีความเสี่ยงต่อการยกเลิกโดยไม่ได้รับอนุญาต. Runtime: Not Executed.
4. ข้อเสนอ	กำหนดวิธีพิสูจน์สิทธิ์ต่อออเดอร์ (เช่น authenticated customer session หรือ token ที่เดาไม่ได้/ผูกกับ order), ตรวจสิทธิ์ที่ server และเก็บ audit; ทดสอบทั้ง allowed/denied path บน test DB.

สถานะ finding: Static source observation + inference; ยังไม่ใช่ exploit ที่พิสูจน์แล้ว. ต้อง review และแก้ตาม requirement จริงก่อน submit.

ขั้นที่ 5 — Test scope and strategy

In scope	ระดับ / strategy
สร้างคำสั่งซื้อและคำนวณยอดรวม	Integration/API + System; ตรวจ response, order, order_items และผลรวมจาก server-side price.
จำนวน/สินค้าไม่ถูกต้อง	Boundary/equivalence: 1, 0, -1, 1.5; product nonexistent/inactive; expected client error/no partial write เป็น proposed AC.
หลายรายการและ unit price	ตรวจ product × quantity หลายรายการ รวม total เทียบข้อมูลจริงในฐานข้อมูล.
วัตถุดิบไม่พอ/recipe	Integration: ตรวจ 409, rollback, status unchanged และ movement; ตรวจ missing recipe coverage.
การยกเลิกและสถานะ	Integration/System: permission, allowed next transition, skipped transition และ history.
Not in scope in this mini-test	UI styling/cross-browser detail ที่ไม่ใช่ประเด็นหลักของ feature นี้; ไม่ใช่การยืนยันว่า UI ไม่มี defect.

Risk ranking

Rank	Risk	Why
1	การยกเลิกออเดอร์ที่ไม่มีการตรวจสิทธิ์ใน route ที่ตรวจพบ	ความเสี่ยง access control; ต้องยืนยันข้อกำหนดและทดสอบก่อนเปิดใช้.
2	จำนวนสินค้าทศนิยม/invalid input	ไม่มี explicit integer check ที่เห็นใน route; อาจผิดพลาดตอน DB insert.
3	สูตรวัตถุดิบไม่ครบหรือ stock deduction ไม่ถูกต้อง	อาจทำให้ stock ในระบบไม่ตรงกับการผลิตจริง.

ผลการทดสอบจริงของ feature นี้ยังไม่มีใน evidence ที่มี จึงไม่ระบุ Pass/Fail สำหรับกรณีข้างต้น.

ขั้นที่ 6 — Entry and Exit criteria

Entry criteria	Exit criteria
1. App และ PostgreSQL test database พร้อม; ใช้ข้อมูลทดสอบที่ไม่ใช่ลูกค้าจริง.	1. ทุกกรณีในชุดทดสอบมีสถานะ actual: Pass/Fail/Blocked/Not Executed และอ้างหลักฐานจริง.
2. มี product/price/recipe/stock ที่ทราบค่าคาดหวัง และมี test accounts/roles ที่จำเป็น.	2. ไม่มี defect ระดับสูงที่ยังไม่ได้ตัดสินใจ; หากมีต้องระบุ blocker/owner/decision.
3. Requirement/AC ที่ใช้มี source/version หรือทำเครื่องหมาย proposed/Not Ready.	3. Re-test หลังแก้ไขและอัปเดต evidence; ไม่อนุญาตให้เปลี่ยน expected result ให้ตรงผลจริง.

ขั้นที่ 7 — Team roles from original worksheet (student IDs removed)

สมาชิก	หน้าที่ที่ระบุในใบงานเดิม	GitHub username
นันทวัฒน์ แซ่ย่าง	Test Scope, Strategy, Risk, Entry/Exit	@Nanthawat67
ธนกร ผดุงศิลป์	Feature และ Business Rule	ยังไม่ให้มา — เติมก่อน submit
อภิรยา สายนาคา	Component และ Data Flow	ยังไม่ให้มา — เติมก่อน submit
ณัฐพงศ์ จันทร์สิงห์	ตรวจสอบกฎและข้อค้นพบ	ยังไม่ให้มา — เติมก่อน submit

AI use (based on the provided worksheet and current drafting): ใช้ AI ช่วยอธิบายโครงสร้าง/จัดข้อความและชี้จุดเสี่ยงจาก code paths; สมาชิกต้องตรวจความถูกต้องและบันทึกข้อเสนอที่รับ/แก้/ปฏิเสธใน AI-LOG.md. ไม่ได้ใช้ AI สร้างผลการรันหรือ screenshots.